

SPEECHMATICS TRACK · AI INFRA SUMMIT HACKATHON 2026

SETTLE

Streaming ASR tells you what it heard.
Nothing tells you when it stopped changing.

Two numbers, and a blind spot in **between** them

Every ASR vendor publishes **word error rate** – was the final text right – and **latency** – how fast did it arrive.

Neither describes the interval where the text is visible, plausible, and **still changing**. Nobody publishes that number.

We call it **settling time**. This deck is the instrument that measures it, and the sidecar that buys it back.

DEFINITION

What settling time is

A word appears the moment the engine has a guess. That guess isn't always the last one.

QUANTITY	CLOCK	DEFINITION
Emission lag	audio	how long before any text for a word appeared
Settling time	wall	how long the text kept changing after it first appeared
Risk window	wall	how long a word stayed visible, still revisable

Hold a word for τ seconds after its last change, and you know it's done moving.

THE PROBLEM, LIVE

A real console just called in a fire that isn't happening

NAIVE

"...working fire me"

ELEVATED – FIRE

SETTLED (T = 0.774S)

– holding –

waiting

Real, unedited recording – runs/09_md1.0.jsonl. The naive console already dispatched. It's about to correct itself.

METHOD

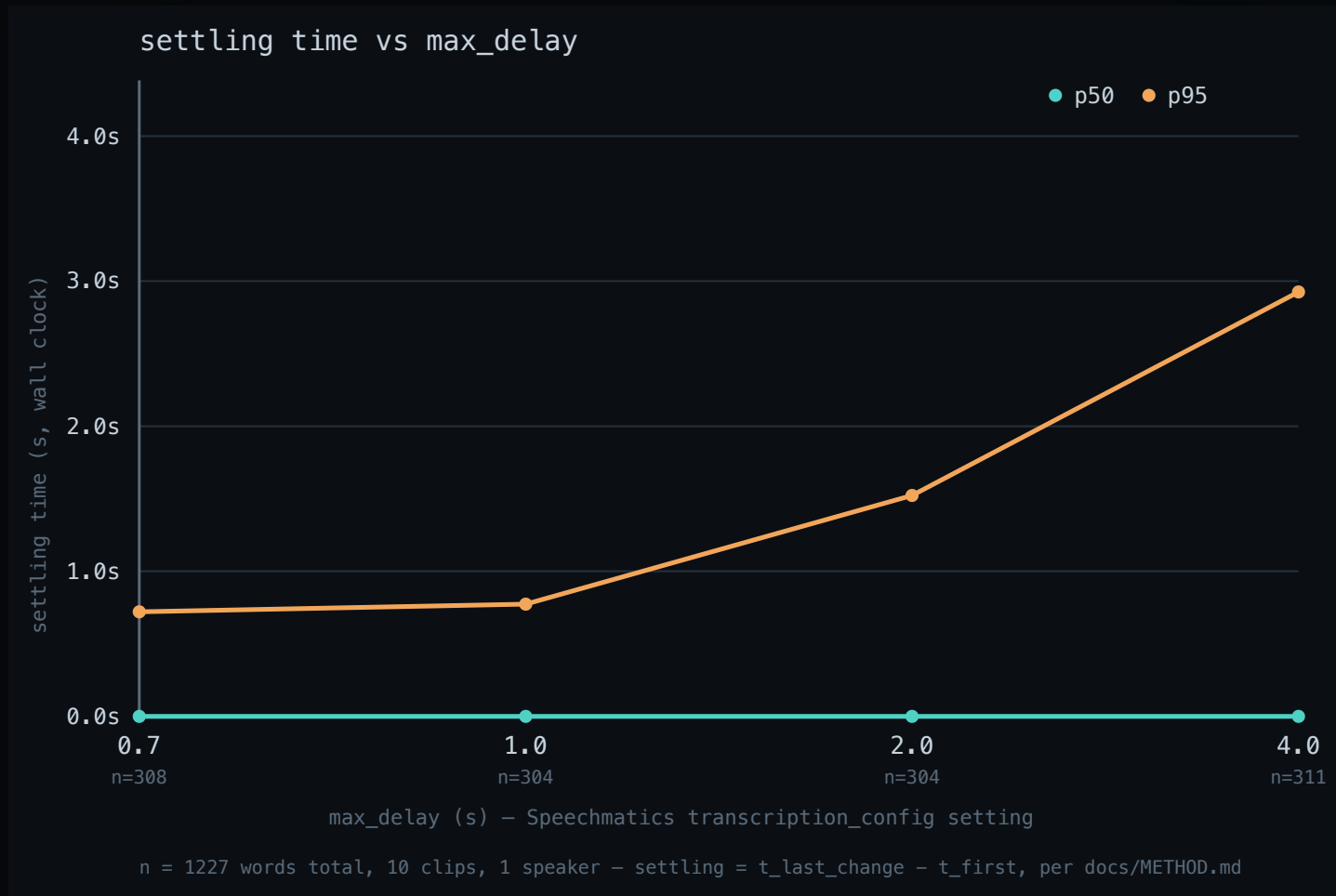
Ten clips. One instrument. Four settings swept.

One speaker, recorded at real-time pace, streamed to Speechmatics and logged with arrival timestamps – never interpreted at capture time.

Every clip recorded at `max_delay` $\in \{0.7, 1.0, 2.0, 4.0\}$ – the engine's own control over when a final is emitted.

Every number that follows traces back to a committed run file. Reproduced with one command: `make results`.

Settling time vs max_delay



RESULTS

The measured table

MAX_DELAY	WORDS	SETTLING P50	SETTLING P99	REVISED	REVISED AFTER FINAL
0.7	308	0.000s	0.985s	32.8%	0.0%
1.0	304	0.000s	1.154s	33.6%	0.0%
2.0	304	0.000s	1.846s	41.4%	0.0%
4.0	311	0.000s	3.441s	45.0%	0.0%

Pasted from make results, never typed.

THE HEADLINE

Final really is final.
It just takes a while to **get** there.

1,227

words observed across the
full sweep

0.0%

changed after their
AddTranscript

3.5x

settling p99 scales from
max_delay 0.7 to 4.0

The danger isn't that finals lie. It's how long a word sits in the open, looking final, before it actually is one.

THE SIDECAR

One feed in. Two streams out.

sidecar.py holds a word until its text has been stable for τ , then releases it once.
One upstream connection, byte-identical input to both consumers.

0.774s

default τ – settling p95 at
max_delay=1.0

THE TRADE

τ has to match its regime

	TOKENS	RESIDUAL RETRACTION	ADDED LATENCY P50	P95
md=1.0 (τ calibrated here)	307	4.2%	0.774s	0.774s
all four max_delay pooled	1,400	13.9%	0.774s	1.734s

The same $\tau = 0.774s$ holds retraction to 2.3% at max_delay=0.7 – and lets it climb to 24.9% at max_delay=4.0. Settling simply takes longer there.

THE DEMO

Same input. One extra second of patience.

NAIVE

ELEVATED – FIRE → STANDARD → CRITICAL – RESCUE

false alarm, fired and reversed

SETTLED

CRITICAL – RESCUE, once

correct, first time

Verified reproducible across repeated live runs against the same replay – runs/09_md1.0.jsonl, make demo.

SPEECHMATICS TRACK · AI INFRA SUMMIT HACKATHON 2026

"The team that measured how long
'final' isn't final."

github.com/adindamochamad/settle · adindamochamad.github.io/settle